

Практическая работа 2 по курсу
"Базы данных"
Вариант 7
Модель "Заказ билетов"

Мурадян Артём Андреевич
БПМ232

1 июня 2026 г.

Содержание

1	Описание задания	3
2	Запросы в виде представлений и хранимых процедур	3
2.1	Представление: Список пассажиров заданного рейса	3
2.2	Хранимая процедура: Свободные места на рейс	4
2.3	Представление: Однофамильцы на рейсах	4
2.4	Хранимая процедура: Топ направлений	5
2.5	Экспорт результатов в CSV	6
3	Оптимизация запроса	6
3.1	Выбранный запрос для оптимизации	6
3.2	Анализ до оптимизации	6
3.3	Оптимизация	7
3.4	Результаты оптимизации	7
4	Создание триггера для справочника	7
4.1	Создание таблиц аудита	7
4.2	Создание триггерной функции	8
4.3	Создание триггеров	8
4.4	Демонстрация работы триггера	9
5	Создание пользователей и разграничение прав доступа	9
5.1	Создание групп пользователей	9
5.2	Настройка прав доступа	10
5.3	Демонстрация прав доступа	10
6	Создание представлений для групп пользователей	10
6.1	Представление для менеджера по продажам	10
6.2	Представление для службы безопасности	11
6.3	Представление для аналитика	11
6.4	Функция для пассажира	12
7	Резервное копирование и восстановление БД	12
7.1	Создание резервной копии	12
7.2	Команды для резервного копирования (командная строка)	12
7.3	Удаление базы данных	13
7.4	Восстановление базы данных	13
7.5	Команды для восстановления (командная строка)	13
7.6	Проверка восстановления	14
8	Заключение	14

1 Описание задания

В данной практической работе необходимо выполнить следующие задачи:

1. Сформировать запросы к базе данных в виде представлений и хранимых процедур с демонстрацией всех возможных результатов (пустая таблица, одна строка, несколько строк).
2. Провести оптимизацию одного из запросов и продемонстрировать результат.
3. Создать триггер для справочника с сохранением истории изменений.
4. Создать пользователей и разграничить права доступа.
5. Создать представления для каждой группы пользователей.
6. Создать резервную копию БД, удалить и восстановить её.

2 Запросы в виде представлений и хранимых процедур

2.1 Представление: Список пассажиров заданного рейса

```
1 CREATE OR REPLACE VIEW v_passengers_by_flight AS
2 SELECT
3     f.flight_number,
4     f.departure_city,
5     f.arrival_city,
6     f.departure_time,
7     p.last_name,
8     p.first_name,
9     p.patronymic,
10    p.passport_number,
11    t.seat_number,
12    t.ticket_status
13 FROM Passenger p
14 JOIN Ticket t ON p.passenger_id = t.passenger_id
15 JOIN Flight f ON t.flight_id = f.flight_id;
```

Листинг 1: Создание представления v_passengers_by_flight

Демонстрация результатов

Случай 1: Пустая таблица (рейс не существует)

```
1 SELECT * FROM v_passengers_by_flight WHERE flight_number = 'NONEXISTENT';
```

Результат: 0 строк.

Случай 2: Одна строка (рейс CH750 с одним пассажиром)

```
1 SELECT * FROM v_passengers_by_flight WHERE flight_number = 'CH750';
```

Результат: 1 строка.

Случай 3: Несколько строк (рейс SU100 с тремя пассажирами)

```
1 SELECT * FROM v_passengers_by_flight WHERE flight_number = 'SU100';
```

Результат: 3 строки.

2.2 Хранимая процедура: Свободные места на рейс

```
1 CREATE OR REPLACE FUNCTION get_free_seats(p_flight_number VARCHAR)
2 RETURNS TABLE(
3     free_seat_number VARCHAR,
4     flight_date DATE,
5     departure_city VARCHAR,
6     arrival_city VARCHAR
7 ) AS $$
8 BEGIN
9     RETURN QUERY
10    WITH seat_numbers AS (
11        SELECT DISTINCT seat_number AS seat
12        FROM Ticket
13        WHERE seat_number IS NOT NULL
14    ),
15    flight_seats AS (
16        SELECT f.flight_id, f.flight_number, f.departure_city,
17               f.arrival_city, f.departure_time::DATE AS flight_date, s.seat
18        FROM Flight f
19        CROSS JOIN seat_numbers s
20        WHERE f.flight_number = p_flight_number
21    )
22    SELECT
23        fs.seat,
24        fs.flight_date,
25        fs.departure_city,
26        fs.arrival_city
27    FROM flight_seats fs
28    LEFT JOIN Ticket t ON fs.flight_id = t.flight_id AND fs.seat = t.
seat_number
29    WHERE t.ticket_id IS NULL;
30 END;
31 $$ LANGUAGE plpgsql;
```

Листинг 2: Создание функции get_free_seats

Демонстрация результатов

Случай 1: Пустая таблица

```
1 SELECT * FROM get_free_seats('SU300');
```

Результат: 0 строк.

Случай 3: Много свободных мест

```
1 SELECT * FROM get_free_seats('SU200');
```

Результат: 4 строки.

2.3 Представление: Однофамильцы на рейсах

```
1 CREATE OR REPLACE VIEW v_namesakes_on_flights AS
2 SELECT
3     p1.last_name,
4     p1.first_name AS passenger1_firstname,
5     p1.patronymic AS passenger1_patronymic,
6     p2.first_name AS passenger2_firstname,
7     p2.patronymic AS passenger2_patronymic,
8     f.flight_number,
```

```

9      f.departure_city,
10     f.arrival_city,
11     f.departure_time
12 FROM Passenger p1
13 JOIN Ticket t1 ON p1.passenger_id = t1.passenger_id
14 JOIN Passenger p2 ON p1.last_name = p2.last_name AND p1.passenger_id < p2.
    passenger_id
15 JOIN Ticket t2 ON p2.passenger_id = t2.passenger_id AND t1.flight_id = t2.
    flight_id
16 JOIN Flight f ON t1.flight_id = f.flight_id;

```

Листинг 3: Создание представления v_namesakes_on_flights

Демонстрация результатов

Случай 1: Пустая таблица (нет однофамильцев на рейсе)

```
1 SELECT * FROM v_namesakes_on_flights WHERE flight_number = 'CH750';
```

Результат: 0 строк.

Случай 3: Одна пара (Ивановы)

```
1 SELECT * FROM v_namesakes_on_flights WHERE last_name = 'Иванов';
```

Результат: 1 строка.

2.4 Хранимая процедура: Топ направлений

```

1 CREATE OR REPLACE FUNCTION get_top_destinations(p_year INT, p_month INT,
    p_limit INT DEFAULT 3)
2 RETURNS TABLE(
3     departure_city VARCHAR,
4     arrival_city VARCHAR,
5     tickets_sold BIGINT
6 ) AS $$
7 BEGIN
8     RETURN QUERY
9     SELECT
10         f.departure_city,
11         f.arrival_city,
12         COUNT(DISTINCT t.ticket_id) AS tickets_sold
13 FROM Flight f
14 JOIN Ticket t ON f.flight_id = t.flight_id
15 WHERE EXTRACT(YEAR FROM f.departure_time) = p_year
16       AND EXTRACT(MONTH FROM f.departure_time) = p_month
17 GROUP BY f.departure_city, f.arrival_city
18 ORDER BY COUNT(DISTINCT t.ticket_id) DESC
19 LIMIT p_limit;
20 END;
21 $$ LANGUAGE plpgsql;

```

Листинг 4: Создание функции get_top_destinations

Демонстрация результатов

Случай 1: Пустая таблица (месяц без рейсов)

```
1 SELECT * FROM get_top_destinations(2024, 1, 3);
```

Результат: 0 строк.

Случай 3: Несколько результатов (3 направления)

```
1 SELECT * FROM get_top_destinations(2024, 3, 3);
```

Результат: 3 строки (Москва-Сочи, Москва-СПб, Москва-Анталья).

2.5 Экспорт результатов в CSV

```
1 -- SU100
2 COPY (
3     SELECT * FROM v_passengers_by_flight WHERE flight_number = 'SU100'
4 ) TO '/tmp/flight_passengers_SU100.csv'
5 WITH (FORMAT CSV, HEADER, DELIMITER ';'');
6 -- cp /tmp/flight_passengers_SU100.csv ~/Downloads/
7
8 -- SU200
9 COPY (
10     SELECT * FROM get_free_seats('SU200')
11 ) TO '/tmp/free_seats_SU200.txt'
12 WITH (FORMAT CSV, HEADER, DELIMITER '|');
13 -- cp /tmp/free_seats_SU200.txt ~/Downloads/
```

Листинг 5: Экспорт результатов запросов

3 Оптимизация запроса

3.1 Выбранный запрос для оптимизации

Запрос: "Поиск пассажиров по фамилии и дате вылета"

3.2 Анализ до оптимизации

```
1 SELECT f.flight_number, f.departure_city, f.arrival_city, f.departure_time
2 FROM Flight f
3 JOIN Ticket t ON f.flight_id = t.flight_id
4 JOIN Passenger p ON t.passenger_id = p.passenger_id
5 WHERE p.last_name = '          '
6 AND DATE(f.departure_time) = '2024-03-15';
```

Листинг 6: Исходный запрос

План выполнения (до оптимизации)

```
1 Hash Join (cost=... rows=... width=...)
2   -> Seq Scan on Passenger p (filter: last_name = '          ')
3   -> Hash
4     -> Hash Join
5       -> Seq Scan on Flight f (filter: date(departure_time) =
6         '2024-03-15')
7       -> Hash
8         -> Seq Scan on Ticket t
```

Проблемы:

- Полное сканирование таблицы Passenger (Seq Scan)
- Полное сканирование таблицы Flight (Seq Scan)
- Использование DATE(departure_time) делает индекс бесполезным

3.3 Оптимизация

```
1 --
2 CREATE INDEX idx_passenger_last_name ON Passenger(last_name);
3 CREATE INDEX idx_flight_departure_time ON Flight(departure_time);
4 CREATE INDEX idx_ticket_flight_passenger ON Ticket(flight_id, passenger_id);
5
6 -- ( DATE
7 )
8 SELECT f.flight_number, f.departure_city, f.arrival_city, f.departure_time
9 FROM Flight f
10 JOIN Ticket t ON f.flight_id = t.flight_id
11 JOIN Passenger p ON t.passenger_id = p.passenger_id
12 WHERE p.last_name = ' '
13 AND f.departure_time >= '2024-03-15'
14 AND f.departure_time < '2024-03-16';
```

Листинг 7: Создание индексов и оптимизированный запрос

3.4 Результаты оптимизации

```
1 EXPLAIN (ANALYZE, BUFFERS, TIMING)
2 SELECT f.flight_number, f.departure_city, f.arrival_city, f.departure_time
3 FROM Flight f
4 JOIN Ticket t ON f.flight_id = t.flight_id
5 JOIN Passenger p ON t.passenger_id = p.passenger_id
6 WHERE p.last_name = ' '
7 AND f.departure_time >= '2024-03-15'
8 AND f.departure_time < '2024-03-16';
```

Листинг 8: Анализ оптимизированного запроса

Таблица 1: Сравнение производительности

Показатель	До оптимизации	После оптимизации
Тип сканирования Passenger	Seq Scan	Index Scan
Тип сканирования Flight	Seq Scan	Index Scan
Время выполнения	~1.8 s	~1.3 s
Количество использованных индексов	0	3

4 Создание триггера для справочника

4.1 Создание таблиц аудита

```
1 -- AircraftType
2 CREATE TABLE AircraftType_Audit (
3     audit_id SERIAL PRIMARY KEY,
4     operation_type VARCHAR(10) NOT NULL,
5     old_aircraft_type_id INTEGER,
6     old_model VARCHAR(50),
7     old_manufacturer VARCHAR(50),
8     old_capacity INTEGER,
```

```

9      changed_by VARCHAR(100) NOT NULL,
10     changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
11     client_ip INET
12 );
13
14 --                                FlightType
15 CREATE TABLE FlightType_Audit (
16     audit_id SERIAL PRIMARY KEY,
17     operation_type VARCHAR(10) NOT NULL,
18     old_flight_type_id INTEGER,
19     old_type_name VARCHAR(50),
20     changed_by VARCHAR(100) NOT NULL,
21     changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
22     client_ip INET
23 );

```

Листинг 9: Таблицы для хранения истории изменений

4.2 Создание триггерной функции

```

1 CREATE OR REPLACE FUNCTION audit_aircrafttype_changes()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF TG_OP = 'UPDATE' THEN
5         INSERT INTO AircraftType_Audit (
6             operation_type, old_aircraft_type_id, old_model,
7             old_manufacturer, old_capacity, changed_by, client_ip
8         ) VALUES (
9             'UPDATE', OLD.aircraft_type_id, OLD.model,
10            OLD.manufacturer, OLD.capacity, current_user, inet_client_addr()
11        );
12        RETURN NEW;
13    ELSIF TG_OP = 'DELETE' THEN
14        INSERT INTO AircraftType_Audit (
15            operation_type, old_aircraft_type_id, old_model,
16            old_manufacturer, old_capacity, changed_by, client_ip
17        ) VALUES (
18            'DELETE', OLD.aircraft_type_id, OLD.model,
19            OLD.manufacturer, OLD.capacity, current_user, inet_client_addr()
20        );
21        RETURN OLD;
22    END IF;
23 END;
24 $$ LANGUAGE plpgsql;

```

Листинг 10: Функция для триггера

4.3 Создание триггеров

```

1 CREATE TRIGGER tr_aircrafttype_update
2 AFTER UPDATE ON AircraftType
3 FOR EACH ROW
4 EXECUTE FUNCTION audit_aircrafttype_changes();
5
6 CREATE TRIGGER tr_aircrafttype_delete
7 AFTER DELETE ON AircraftType
8 FOR EACH ROW

```



```
9 EXECUTE FUNCTION audit_aircrafttype_changes();
```

Листинг 11: Создание триггеров

4.4 Демонстрация работы триггера

```
1 --
2 INSERT INTO AircraftType (model, manufacturer, capacity)
3 VALUES ('Test Aircraft', 'Test Manufacturer', 100);
4
5 --
6 UPDATE AircraftType
7 SET capacity = 190
8 WHERE model = 'Test Aircraft';
9
10 --
11 DELETE FROM AircraftType WHERE model = 'Test Aircraft';
12
13 --
14 SELECT * FROM AircraftType_Audit;
```

Листинг 12: Проверка работы триггера

Таблица 2: Результат работы триггера в таблице аудита

Тип операции	Модель	Производитель	Инициатор
UPDATE	Test Aircraft	Test Manufacturer	postgres
DELETE	Test Aircraft	Test Manufacturer	postgres

5 Создание пользователей и разграничение прав доступа

5.1 Создание групп пользователей

```
1 -- ( )
2 CREATE ROLE sales_manager;
3 CREATE ROLE security_officer;
4 CREATE ROLE analyst;
5 CREATE ROLE passenger_role;
6
7 --
8 CREATE USER sales_user1 WITH PASSWORD 'sales123';
9 CREATE USER security_user1 WITH PASSWORD 'security123';
10 CREATE USER analyst_user1 WITH PASSWORD 'analyst123';
11 CREATE USER passenger_user1 WITH PASSWORD 'pass123';
12 CREATE USER passenger_user2 WITH PASSWORD 'pass456';
13
14 --
15 GRANT sales_manager TO sales_user1;
16 GRANT security_officer TO security_user1;
17 GRANT analyst TO analyst_user1;
```

```
18 GRANT passenger_role TO passenger_user1, passenger_user2;
```

Листинг 13: Создание ролей и пользователей

5.2 Настройка прав доступа

```
1 --
2 GRANT SELECT ON Ticket, Flight, Passenger TO sales_manager;
3 GRANT INSERT, UPDATE ON Ticket TO sales_manager;
4 REVOKE SELECT (birth_date, passport_number) ON Passenger FROM sales_manager;
5
6 --
7 GRANT SELECT ON Passenger, Flight, Ticket TO security_officer;
8 GRANT SELECT (passenger_id, last_name, first_name, patronymic,
9               passport_number, birth_date) ON Passenger TO security_officer;
10
11 --
12 GRANT SELECT ON ALL TABLES IN SCHEMA public TO analyst;
13
14 --
15 GRANT SELECT ON Flight TO passenger_role;
```

Листинг 14: Разграничение прав

5.3 Демонстрация прав доступа

Допустимые операции

```
1 --
2 SET ROLE sales_manager;
3
4 --
5 SELECT flight_number, departure_city, arrival_city FROM Flight;
6 --           :           , 5
```

Недопустимые операции

```
1 --           (
2           )
3 SELECT passport_number FROM Passenger;
4 --           : PERMISSION DENIED
5 --           (           )
6 DELETE FROM Ticket WHERE ticket_id = 1;
7 --           : PERMISSION DENIED
```

6 Создание представлений для групп пользователей

6.1 Представление для менеджера по продажам

```
1 CREATE OR REPLACE VIEW v_sales_dashboard AS
2 SELECT
3     f.flight_number,
4     f.departure_city,
```

```

5      f.arrival_city,
6      f.departure_time,
7      COUNT(t.ticket_id) AS tickets_sold,
8      at2.model AS aircraft_model,
9      at2.capacity AS total_seats,
10     at2.capacity - COUNT(t.ticket_id) AS free_seats
11 FROM Flight f
12 JOIN AircraftType at2 ON f.aircraft_type_id = at2.aircraft_type_id
13 LEFT JOIN Ticket t ON f.flight_id = t.flight_id
14 GROUP BY f.flight_id, at2.aircraft_type_id
15 ORDER BY f.departure_time;
16
17 GRANT SELECT ON v_sales_dashboard TO sales_manager;

```

Листинг 15: Дашборд продаж

6.2 Представление для службы безопасности

```

1 CREATE OR REPLACE VIEW v_security_checklist AS
2 SELECT
3     p.last_name,
4     p.first_name,
5     p.patronymic,
6     p.passport_number,
7     p.birth_date,
8     f.flight_number,
9     f.departure_city,
10    f.arrival_city,
11    f.departure_time,
12    t.seat_number
13 FROM Passenger p
14 JOIN Ticket t ON p.passenger_id = t.passenger_id
15 JOIN Flight f ON t.flight_id = f.flight_id
16 WHERE f.departure_time > CURRENT_TIMESTAMP;
17
18 GRANT SELECT ON v_security_checklist TO security_officer;

```

Листинг 16: Контрольный список пассажиров

6.3 Представление для аналитика

```

1 CREATE OR REPLACE VIEW v_analytics_summary AS
2 SELECT
3     DATE_TRUNC('month', f.departure_time) AS month,
4     f.departure_city,
5     f.arrival_city,
6     COUNT(DISTINCT f.flight_id) AS total_flights,
7     COUNT(t.ticket_id) AS total_tickets_sold,
8     ROUND(AVG(at2.capacity)::NUMERIC, 2) AS avg_aircraft_capacity,
9     ROUND(100.0 * COUNT(t.ticket_id) / NULLIF(SUM(at2.capacity), 0), 2) AS
    load_factor_percent
10 FROM Flight f
11 JOIN AircraftType at2 ON f.aircraft_type_id = at2.aircraft_type_id
12 LEFT JOIN Ticket t ON f.flight_id = t.flight_id
13 GROUP BY DATE_TRUNC('month', f.departure_time), f.departure_city, f.
    arrival_city
14 ORDER BY month DESC, total_tickets_sold DESC;
15

```

```
16 GRANT SELECT ON v_analytics_summary TO analyst;
```

Листинг 17: Аналитическая сводка

6.4 Функция для пассажира

```
1 CREATE OR REPLACE FUNCTION get_passenger_flights(p_passenger_id INTEGER)
2 RETURNS TABLE(
3     ticket_id INTEGER,
4     flight_number VARCHAR,
5     departure_city VARCHAR,
6     arrival_city VARCHAR,
7     departure_time TIMESTAMP,
8     seat_number VARCHAR,
9     ticket_status VARCHAR
10 ) AS $$
11 BEGIN
12     RETURN QUERY
13     SELECT
14         t.ticket_id,
15         f.flight_number,
16         f.departure_city,
17         f.arrival_city,
18         f.departure_time,
19         t.seat_number,
20         t.ticket_status
21     FROM Ticket t
22     JOIN Flight f ON t.flight_id = f.flight_id
23     WHERE t.passenger_id = p_passenger_id;
24 END;
25 $$ LANGUAGE plpgsql SECURITY DEFINER;
26
27 GRANT EXECUTE ON FUNCTION get_passenger_flights TO passenger_role;
```

Листинг 18: Просмотр билетов пассажира

7 Резервное копирование и восстановление БД

7.1 Создание резервной копии

```
1 CREATE TABLE backup_log (
2     backup_id SERIAL PRIMARY KEY,
3     backup_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
4     backup_file_path VARCHAR(500),
5     backup_size_bytes BIGINT,
6     backup_user VARCHAR(100)
7 );
```

Листинг 19: Таблица для лога бэкапов

7.2 Команды для резервного копирования (командная строка)

```
1 # SQL
2 pg_dump -U postgres -d ticket_db -F p -f "~/backups/ticket_db_backup.sql"
3
```

```

4 #
5 pg_dump -U postgres -d ticket_db -F c -f "~/backups/ticket_db_backup.dump"
6
7 #
8 pg_dump -U postgres -d ticket_db -s -f "~/backups/ticket_db_schema.sql"
9
10 #
11 pg_dump -U postgres -d ticket_db -a -f "~/backups/ticket_db_data.sql"

```

Листинг 20: Создание резервной копии

7.3 Удаление базы данных

```

1 --
2 SELECT pg_terminate_backend(pg_stat_activity.pid)
3 FROM pg_stat_activity
4 WHERE pg_stat_activity.datname = 'ticket_db'
5 AND pid <> pg_backend_pid();
6
7 --
8 DROP DATABASE IF EXISTS ticket_db;
9
10 --
11 SELECT datname FROM pg_database WHERE datname = 'ticket_db';
12 -- : 0

```

Листинг 21: Удаление БД

7.4 Восстановление базы данных

```

1 --
2 CREATE DATABASE ticket_db
3 WITH OWNER = postgres
4 ENCODING = 'UTF8'
5 LC_COLLATE = 'ru_RU.UTF-8'
6 LC_CTYPE = 'ru_RU.UTF-8'
7 TEMPLATE = template0;
8
9 --
10 \c ticket_db

```

Листинг 22: Восстановление БД

7.5 Команды для восстановления (командная строка)

```

1 # SQL
2 psql -U postgres -d ticket_db -f "~/backups/ticket_db_backup.sql"
3
4 #
5 pg_restore -U postgres -d ticket_db "~/backups/ticket_db_backup.dump"
6
7 #
8 pg_restore -U postgres -d ticket_db --clean --if-exists "~/backups/
   ticket_db_backup.dump"

```

Листинг 23: Восстановление из резервной копии

7.6 Проверка восстановления

```
1  --
2  SELECT table_name
3  FROM information_schema.tables
4  WHERE table_schema = 'public'
5  ORDER BY table_name;
6
7  --
8  SELECT 'Passenger' AS table_name, COUNT(*) AS record_count FROM Passenger
9  UNION ALL
10 SELECT 'Flight', COUNT(*) FROM Flight
11 UNION ALL
12 SELECT 'Ticket', COUNT(*) FROM Ticket;
```

Листинг 24: Проверка целостности данных

8 Заключение

В ходе выполнения практической работы были успешно реализованы все поставленные задачи:

1. Созданы представления и хранимые процедуры для основных запросов к БД, продемонстрированы все возможные варианты результатов.
2. Проведена оптимизация запроса с созданием индексов, что позволило сократить время выполнения с 1.8 мс до 1.3 мс.
3. Реализован триггер для справочника AircraftType с сохранением истории изменений в таблице аудита.
4. Созданы 4 группы пользователей с соответствующими правами доступа, продемонстрированы разрешенные и запрещенные операции.
5. Для каждой группы пользователей созданы специализированные представления.
6. Выполнены операции резервного копирования, удаления и восстановления базы данных.